

Windows 視窗化的 SEA 演算法具體實現

謝惠婷、楊中皇

國立高雄師範大學 資訊教育研究所

<http://crypto.nknu.edu.tw/>

angle-69@yahoo.com.tw chyang@nknucc.nknu.edu.tw

摘要

美國國家標準技術局(NIST)在 FIPS 186-2 標準文件裡推薦了五組質數體的橢圓曲線參數提供安全的橢圓曲線密碼系統使用，不過若能讓使用者自行隨機產生質數階橢圓曲線參數，則能達到更佳的安全性，其中一個重要的環節是計算橢圓曲線的階數並找到質數階的橢圓曲線參數，而 SEA 演算法(Schoof-Elkies-Atkin algorithm)是計算質數體橢圓曲線階數的有效演算法，以此來尋找質數階的橢圓曲線參數，進而建立安全的橢圓曲線密碼系統。為了實現此演算法，本研究使用 MIRACL 為底層的函式庫，再利用 C++ Builder 設計視窗化執行介面，在 Pentium 1.83G 的處理器上搭配 1G 記憶體尋找可供橢圓曲線密碼系統使用的質數階橢圓曲線參數，基域為 192、224、256、384、521 位元平均分別需要 580、1360、1884、20907、65862 秒可搜尋到一條質數階的橢圓曲線參數，且計算一條橢圓曲線的階數平均分別需要 31.6、56.7、99.4、722.3、2726.0 秒。

關鍵詞：橢圓曲線密碼系統、SEA 演算法(Schoof-Elkies-Atkin algorithm)、模多項式、密碼學(Cryptography)

1. 前言

在 1985 年 Neil Koblitz[10]和 Victor Miller [11]兩位學者分別獨立提出橢圓曲線密碼系統(Elliptic Curve Cryptosystem, ECC)，可用於加解密、數位簽章、金鑰交換及大數分解，因為在相同的安全性下，其金鑰長度小於其他公鑰密碼系統，使其處理速度較快、儲存空間較小而有發展上的優勢。目前 ECC 已被廣泛地制定於各國標準中，如 ISO 1177-3、ANSI X9.62、X9.63、IEEE P1363、FIPS 186-2[5]。

橢圓曲線的安全性是基於橢圓曲線離散對數問題(Elliptic Curve Discrete Logarithm Problem, ECDLP)的困難度，故尋找質數階的橢圓曲線參數對安全性有其重要性。目前處理此問題的方法有兩種類型：

- (1) 複乘方法(complex multiplication)：先確定橢圓曲線的階數，接著尋找合適的橢圓曲線，最後再確認大質數階的點。複乘方法雖然較簡單，實現速度也比較快，但卻存在潛在的威脅[4]。
- (2) 隨機選取：隨機選取橢圓曲線參數並計算它的階，直到找到質數階的橢圓曲線參數

[6]。此類方法可分為 l -adic 求階方法和 p -adic 求階方法二種， l -adic 方法(SEA 演算法)在特徵值為大質數時較有效率 [3,6]，且有下列二種演算法：

- Schoof 演算法：R. Schoof 於 1985 年首先提出此演算法，此演算法的時間複雜度為 $O(\log^8 q)$ [3]，並使用 Forbenius 自同態和除子多項式，但因除子多項式的次數太高，故需大量的儲存空間，因而並不實用。
- SEA 演算法：為增進 Schoof 演算法的效率，Elkies 和 Atkin 提出改進，因此稱為 SEA(Schoof-Elkies-Atkin)演算法，時間複雜度為 $O(\log^6 q)$ [3]。後來還有 Morain 提出使用同種圈(Isogeny cycle)的方法，及 Couveignes、Lercier 等人對演算法進行優化以改進效率 [4,6]，使得 SEA 演算法對於建立安全橢圓曲線密碼系統更具價值。

2. 文獻探討

2.1 橢圓曲線離散對數問題(Elliptic Curve Discrete Logarithm Problem, ECDLP)

F_q 下給定一條橢圓曲線 E 及二個點 $P, Q \in E$ ，如果 x 存在，則找到一個 $x \in Z$ ，使得 $Q = [x]P$ ，此為橢圓曲線離散對數問題[10]。橢圓曲線密碼系統的安全性乃基於 ECDLP 的困難度，為了確保系統的安全，所選取的曲線要能避免已知對橢圓曲線的攻擊，如 baby-step giant-step(小步—大步)方法、Pollard ρ 方法、Pollard λ 方法、Pohlig-Hellman 方法、MOV 攻擊法等。

2.1.1 Shank's Method(小步—大步演算法，baby-step giant-step, BSGS)

由 Shanks 在 1971 年提出，用來尋找 $m = \lceil \sqrt{n} \rceil a + b$ 使得 $Q = [m]P$ ，其中 $P, Q \in G$ 且 $0 \leq a, b < \lceil \sqrt{n} \rceil - 1$ 。若 $Q = [\lceil \sqrt{n} \rceil a + b]P$ ，經由移項可得 $Q - [b]P = [a](\lceil \sqrt{n} \rceil P)$ ，其中 $R_b = Q - [b]P$ 稱為小步(baby-step)，並將其計算

結果儲存於一張表格中； $S_a = [a]([\sqrt{n}])P$ 稱為大步(giant-step)，在此的每一步計算結果都需在 baby-step 的表格中尋找是否有相同的值。若存在相同的值，則表示找到符合的 a, b 用此計算 $m[8,13]$ 。

2.1.2 MOV 算法

1993 年，Menezes、Okamoto、Vanstone 將橢圓曲線 $E(F_q)$ 上的離散對數問題轉換至一般 F_{q^n} 上的離散對數問題，提出 MOV 算法[3]。但這方法只能在 B 不是很大時才可以有效攻擊，所以測試隨機選取的橢圓曲線是否可以抵擋 MOV 攻擊，也就是測試 B 是否小於某個數值，此值稱為 MOV 條件的門檻值(thresholds)[2]，見表 1。

表 1 MOV 條件的門檻值(thresholds)[9]

q 的長度(bit)	B	q 的長度(bit)	B	q 的長度(bit)	B
128-142	6	281-297	14	407-420	22
143-165	7	29-313	15	421-434	23
166-186	8	314-330	16	435-448	24
187-206	9	331-346	17	449-462	25
207-226	10	347-361	18	463-475	26
227-244	11	362-376	19	476-488	27
245-262	12	377-391	20	489-501	28
263-280	13	392-406	21	502-512	29

演算法：MOV 算法[9]

輸入：MOV 門檻值；prime-power q ；prime r
輸出：True—MOV 條件滿足橢圓曲線 $GF(q)$ 上秩為 r 的基點
False—otherwise

Set $t \leftarrow 1$
For i from 1 to B do:
Set $t \leftarrow tq \bmod r$
If $t=1$, then output “False” and stop
Output “True”

2.2 Schoof 演算法

2.2.1 Hasse 定理

在 F_q 下，橢圓曲線 $E: y^2 = x^3 + ax + b$ 的點數 $\#E(F_q) = q + 1 - t$ ，且 $|t| \leq 2\sqrt{q}$ 。

2.2.2 撓點(Torsion point)

K 是一個 F_q 有限體， $E(\overline{K})$ 是撓群(Torsion group)，也就是在曲線 E 上的所有點都是有限

秩。 E 上的 m 階撓點表為 $E(m)$ ，定義為 $E(m) = \{P \in E(\overline{K}) \mid [m]P = O\}$ [8]。

[定理] P 是 $E(\overline{K}) \setminus O$ 的一點，且令 $m \geq 1$ ，則 $P \in E(m) \Leftrightarrow \psi_m(P) = 0$ [8]。

而 $E(m)$ 中所有非 O 點的 x 坐標為根的多項式，稱為除子多項式[1]。

2.2.3 除子多項式(Division polynomial)

在 F_q 上的橢圓曲線 $E: y^2 = x^3 + ax + b$ ，令

$\psi_m \in F_q[x, y]$ ，定義除子多項式

$$\psi_0 = 0$$

$$\psi_1 = 1$$

$$\psi_2 = 2y$$

$$\psi_3 = 3x^4 + 6ax^2 + 12bx - a^2$$

$$\psi_4 = 4y(x^6 + 5ax^4 + 20bx^3 - 5a^2x^2 - 4abx - 8b^2 - a^3)$$

$$\psi_{2m+1} = \psi_{m+2}\psi_m^3 - \psi_{m-1}\psi_{m+1}^3, \quad m \geq 2$$

$$\psi_{2m} = (2y)^{-1}(\psi_m)(\psi_{m+2}\psi_{m-1}^2 - \psi_{m-2}\psi_{m+1}^2), \quad m \geq 2 \quad [8]。$$

2.2.4 Forbenius 自同態(Forbenius endomorphism)

ϕ 表示方程式 $E: y^2 = x^3 + ax + b$ 在 F_q 的代數閉域 $\overline{F_q}$ 上的 Frobenius 自同態，即 ϕ 表示下列映射：

$\phi: (x, y) \rightarrow (x^q, y^q)$ ，其中 $x, y \in \overline{F_q}$ 且滿足 E 則 t 是滿足同態方程式 $\phi^2 - t\phi + p = 0$ 的唯一整數，且 $|t| \leq 2\sqrt{p}$ 是 Frobenius 同態 ϕ 的跡，則曲線 $E(F_p)$ 上點的個數 $\#E(F_p) = p + 1 - t$ [4]。

2.2.5 Schoof 演算法的想法

演算法：Schoof 演算法[8]

輸入：在有限體 F_q 下的橢圓曲線 E

輸出： $E(F_q)$ 的點數

$M \leftarrow 2, l \leftarrow 3, S \leftarrow \{((t \bmod 2), 2)\}$

While $M < 4\sqrt{q}$ do:

For $\tau = 0, \dots, \frac{l-1}{2}$ do:

for $P \in E[l]$ 檢驗確實存在一個 τ 可以通過 $\phi^2(P) + [q]P = \pm[\tau]\phi(P)$ 測試

$S \leftarrow S \cup \{(\tau, l)\}$ 或 $S \leftarrow S \cup \{(-\tau, l)\}$

$M \leftarrow M \times l$

$l \leftarrow \text{nextprime}(l)$

搜尋足夠多的集合 S 後，利用 CRT(中國剩餘定理，Chinese Remainder Theorem)來整合資訊，計算出 t

Return $\#E(F_q) \leftarrow q + 1 - t$

Schoof 演算法的主要想法是搜集一個質數所成的集合 $S=\{2, 3, \dots, L\}$ ，使得 $\prod_{l \in S} l > 4\sqrt{q}$ ，對每一個 S 中的質數 l ，計算 $t \bmod l$ ，並利用中國剩餘定理計算 t ，最後得橢圓曲線點數 $\#E(F_q) = q+1-t$ [2]。

2.3 SEA 演算法(Schoof-Elkies-Atkin algorithm)

演算法：SEA 演算法[8]
輸入：在有限體 F_q 下的橢圓曲線 E
輸出： $E(F_q)$ 的點數
$M \leftarrow 1, l \leftarrow 2, A \leftarrow \{\}$ 和 $E \leftarrow \{\}$ While $M < 4\sqrt{q}$ do: 使用約化模多項式 $\Phi_l^c(x, y)$ 來判斷質數 l 為 Elkies prime 還是 Atkin prime If l 是 Elkies prime then do: - 計算同種橢圓曲線及同種映射的核 - 得到除子多項式的因式 h_l ， $\deg h_l = (l-1)/2$ (確定多項式 $F_l(x)$) - 尋找 φ 的特徵值 $\lambda \pmod{l}$ - $t \leftarrow \lambda + q/\lambda \pmod{l}$ //唯一值 - $E \leftarrow E \cup \{(t, l)\}$ Else do: // l 是 Atkin prime - 確定候選值集合 T ，使得 $t \pmod{l} \in T$ - $A \leftarrow A \cup \{(T, l)\}$ $M \leftarrow M \times l$ $l \leftarrow \text{nextprime}(l)$ 搜尋足夠多的集合 A 和 E 後，利用 CRT(中國剩餘定理，Chinese Remainder Theorem)和 BSGS(小步—大步，baby-step giant-step)來整合資訊，計算出 t Return $\#E(F_q) \leftarrow q+1-t$

2.3.1 Elkies 質數和 Atkin 質數

Frobenius 變換 φ 的特徵多項式為 $x^2 - tx + p$ ，若在 F_l 中有根，則質數 l 稱為 Elkies 質數，否則質數 l 稱為 Atkin 質數。

[推論] 模多項式 $\Phi_l(x, j(E)) \pmod{p}$ 在 F_p 中有根若且唯若 l 為 Elkies 質數，也就是 $\gcd(x^p - x, \Phi_l(x, j(E)))$ 的次數 ≥ 1 [1]。

因此利用質數 l 的模多項式(Modular polynomial)可以判斷質數 l 是 Elkies 質數或 Atkin 質數，故模多項式在 SEA 演算法中扮演相當重要的角色。

2.3.2 模多項式(Modular polynomial)

模多項式是二元整係數的多項式，可以分為二類：

(1) 經典意義下的模多項式(Classical Modular polynomial) $\Phi_l(X, Y) \in \mathbb{Z}[X, Y]$ ：其 X 和 Y 的次數都是 $l+1$ ，且係數會隨 l 的增加而快速增大，因此在計算和儲存上較沒效率，故 SEA 演算法不採用此模多項式[6]。

(2) 約化模多項式(Reduced Modular polynomial) $\Phi_l^c(x, y) \in \mathbb{Z}[X, Y]$ ：其 X 的次數是 $l+1$ ， Y 的次數是 $\frac{l-1}{\gcd(l-1, 12)}$ ，且係數相對較小，並有將近

一半的係數為零，再加上性質與經典意義下的模多項式相似[1,6,7]，故 SEA 演算法中使用約化模多項式來判斷質數 l 是 Elkies 質數或 Atkin 質數。

- 模多項式的非零係數個數：設 l 為質數， s 是使得 $v = \frac{s \cdot (l-1)}{12}$ 為整數的最小正整數，則 $\Phi_l^c(x, y)$ 共有 $\frac{(l+1)(v+1)}{2} + 2$ 個非零係數[6]，其中 x^{l+1} 項的係數為 1。
- 模多項式的係數特徵：設 l_1 和 l_2 為兩個相近的質數且對應的 s 為 s_1 和 s_2 ，則模多項式的最大係數與 s 近似成正比，且若 $s_1 > s_2$ ，則 $\Phi_{l_1}^c(X, Y)$ 的係數一般會大於 $\Phi_{l_2}^c(X, Y)$ 的係數，這會使得計算上更為費時。因此，拿 s 來判斷模多項式計算上的困難度，在質數 l 較大時，捨棄困難度 $s = 6$ 和 $s = 3$ 的質數 l 。[6]

2.4 SEA 演算法的優化

2.4.1 基域的選取

橢圓曲線密碼系統的運算有加減乘除和求逆運算，其中速度最慢的是除法和求逆運算，若基域選用特殊形式 p ，形如：

- (1) $p = 2^k \pm \alpha$ ， α 是很小的整數
- (2) $p = 2^k \pm 2^s \pm 1$
- (3) $p = 2^k \pm 2^s \pm 2^t \pm 2^u - 1$

則在計算 $B = A \bmod p$ ， $0 \leq A < p^2$ 時，就不需做大整數的除法，只要少量的加減法即可[5,14]，以此來加速運算卻又不影響其安全性[1]，此特殊形式 p 稱為廣義梅森數(generalized Mersenne number)。

2.4.2 提早終止策略

採用隨機選取方式來尋找質數階的橢圓曲

線參數需先隨機選取參數 a 和 b ，再計算曲線的階數，並判斷是否為質數，如果不是，則需重新選取新的參數，再重複上述步驟，直到得到一條質數階的曲線為止。由 Hasse 定理可知橢圓曲線的點數 $\#E(F_p) = p+1-t$ ，若在計算曲線的階數前，即可判斷該曲線的階數不是質數，表示可以找到 l 整除 $p+1-t$ ，則沒有必要計算出該曲線的階數，而提早終止此曲線階數的計算，直接重新選取下一組曲線參數。

如果方程式 $x^3 + ax + b \equiv 0 \pmod{p}$ 有根 α ，則表示 $E(F_p)$ 的有理點 $(\alpha, 0)$ 的階為 2，也就是此橢圓曲線的階數為偶數。而解方程式 $x^3 + ax + b \equiv 0 \pmod{p}$ 實際上就是計算 $\gcd(x^p - x, x^3 + ax + b)$ ，而 $\gcd(x^p - x, x^3 + ax + b) = 1$ ，表示方程式 $x^3 + ax + b \equiv 0 \pmod{p}$ 有根，也就是橢圓曲線的階數為偶數，其中隨機選取的橢圓曲線，有大約一半的橢圓曲線的階數為偶數[6]。

2.4.3 倍點及點加的計算優化

用 (x, y) 表示橢圓曲線上的點，稱為仿射座標(Affine Coordinate)。表 2、表 3 為仿射座標下分析質數體橢圓曲線 $y^2 = x^3 + ax + b \pmod{p}$ 其點加和倍點的公式和運算量，因為加減法的運算量較小，故忽略不計[1]，僅考慮乘法運算、平方運算、求逆運算，且使用 M、S、I 來表示。

表 2 仿射座標下，質數體點加運算公式[1]
點加 $(x_0, y_0) + (x_1, y_1) = (x_2, y_2)$

公式	運算量
$\lambda = \frac{x_1 - x_0}{y_1 - y_0}$	1M+1I
$x_2 = \lambda^2 - x_0 - x_1$	1S
$y_2 = \lambda(x_0 - x_2) - y_0$	1M
總運算量 共計	2M+1S+1I

表 3 仿射座標下，質數體倍點運算公式[1]
倍點 $2(x_1, y_1) = (x_2, y_2)$

公式	運算量
$\lambda = \frac{3x_1^2 + a}{2y_1}$	1M+1S+1I
$x_2 = \lambda^2 - 2x_1$	1S
$y_2 = \lambda(x_1 - x_2) - y_1$	1M
總運算量 共計	2M+2S+1I

從表 2、表 3 發現質數體的點加及倍點運算皆使用到一次求逆運算，這相當費時，因此可以將座標轉換成投射座標(Projective Coordinate)或 Jacobian 座標以達到降低運算量的目的。

在 MIRACL[12]中原始碼 sea.cpp 也採用轉

換座標成 Jacobian 座標，以達到減少運算量並增加效能，表 4 為 IEEE P1363 標準[9]中 Jacobian 座標下點加的運算量整理，算法中令座標 $z_1 = 1$ ，使得 $U_0 = x_0$ ， $S_0 = y_0$ ，可以再減少 3M+1S 的運算量，使得總運算量降低至 8M+3S。

表 4 Jacobian 座標下點加的運算量[1,9]

$(x_0, y_0, z_0) + (x_1, y_1, z_1) = (x_2, y_2, z_2)$		
公式	運算量	$z_1 = 1$
$U_0 = x_0 z_1^2$	1M+1S	
$S_0 = y_0 z_1^3$	2M	
$U_1 = x_1 z_0^2$	1M+1S	1M+1S
$S_1 = y_1 z_0^3$	2M	2M
$W = U_0 - U_1$		
$R = S_0 - S_1$		
$T = U_0 + U_1$		
$M = S_0 + S_1$		
$z_2 = z_0 z_1 W$	2M	1M
$x_2 = R^2 - TW^2$	1M+2S	1M+2S
$V = TW^2 - 2x_2$		
$2y_2 = VR - MW^3$	3M	3M
總運算量 共計	12M+4S	8M+3S

另外，表 5 為 IEEE P1363 標準[9]中 Jacobian 座標下倍點的運算量整理，值得注意的是橢圓曲線的參數 a ，如果令 $a = -3$ ，在計算 $M = 3x_1^2 + az_1^4$ 時，因為 $M = 3(x_1 + z_1^2)(x_1 - z_1^2)$ ，可使運算量由 1M+3S 減少到 1M+1S，且總運算量只需要 4M+4S。

表 5 Jacobian 座標下倍點的運算量[1,9]

$2(x_1, y_1, z_1) = (x_2, y_2, z_2)$		
公式	運算量	$a = -3$
$M = 3x_1^2 + az_1^4$	1M+3S	1M+1S
$z_2 = 2y_1 z_1$	1M	1M
$S = 4x_1 y_1^2$	1M+1S	1M+1S
$x_2 = M^2 - 2S$	1S	1S
$T = 8y_1^4$	1S	1S
$y_2 = M(S - x_2) - T$	1M	1M
總運算量 共計	4M+6S	4M+4S

表 6 點加及倍點運算量比較表

	點加	倍點
A：仿射座標	2M+1S+1I	2M+2S+1I
P：標準投射座標	12M+2S	7M+3S
J：Jacobian 座標	12M+4S	4M+6S
	$z_1 = 1$ 時， 降為 8M+3S	$a = -3$ 時， 降為 4M+4S

最後，表 6 為三種座標下點加及倍點運算量的比較表。

3. 系統實作

3.1 MIRACL(Multiprecision Integer and Rational Arithmetic C/C++ Library)簡介

MIRACL[12]是多倍精度的大數函式庫，由 Shamus Software Ltd.開發，目前版本為 5.3.2 版，可操作於 Windows(DOS)和 Linux 作業系統。MIRACL 對於橢圓曲線求階問題提供了複乘方法、Schoof 演算法和 SEA 演算法等方法，而本研究採用 MIRACL 裡關於 SEA 演算法的相關原始碼來開發系統以進行橢圓曲線點數的計算。

3.2 系統設計與架構

MIRACL[12]裡提供關於實現 SEA 演算法的相關開放原始碼及程式，本研究使用大數函式庫 MIRACL 並利用 Borland C++ Builder 6.0 設計視窗執行介面。圖 1 為系統實作的流程圖，首先依據模多項式的係數特徵選擇困難度較小的模多項式，執行 mueller.exe 來生成，接著選擇為廣義梅森數的基域，再使用 process.exe 針對不同的基域做前置處理，最後才能提供 sea.exe 用來計算橢圓曲線的點數，搭配質數的判斷來隨機產生質數階的橢圓曲線參數，供橢圓曲線密碼系統使用。

當基域較大時，常需要使用較多的模多項式才能計算出橢圓曲線的點數，因此搜集足夠多的模多項式對於實現 SEA 演算法是非常重要的前置工作。觀察模多項式的係數特徵發現模

多項式的最大係數與困難度 s 有近似正比的關係[6]，而較大的係數會使得計算量增加，進而影響計算的效率，因此在挑選模多項式時，參考模多項式的困難度 s ，選擇困難度小的質數，在 150 之前的質數，由於係數都較小，所以全部選用；在 150 到 300 之間的質數，捨棄 $s=6$ 的質數；在 300 到 400 之間的質數，捨棄掉 $s=3$ 的質數；最後在 400 到 601 之間的質數就只留下 $s=1$ 的質數。最後本研究選擇以下 70 個質數來產生模多項式：3、5、7、11、13、17、19、23、29、31、37、41、43、47、53、59、61、67、71、73、79、83、89、97、101、103、107、109、113、127、131、137、139、149、151、157、163、173、181、193、197、199、211、223、229、233、241、257、269、271、277、281、283、293、307、313、331、337、349、367、373、379、397、409、421、433、457、541、577、601。但在計算基域為 521 位元($p=2^{521}-1$)的橢圓曲線點數時，仍有可能發生模多項式不足使用的情形，這會使得最後在資訊整合時發生困難，因此再加入質數 167、179、317、439 的模多項式供其使用。

表 7 為本研究系統實作選擇使用的基域，皆為廣義梅森數形式，其中 p192-4、p224-1、p256-5、p384-1、p521-1 為美國國家標準技術局(NIST)在 FIPS 182-6 標準中推薦的五組質數體橢圓曲線參數所使用的基域。

由於大部分橢圓曲線的點數並非質數，並不能提供安全的橢圓曲線密碼系統使用，故使用提早終止策略，當能明確判斷橢圓曲線的點數不是質數時，就直接放棄計算這條曲線的點數，而本系統會固定參數 A 且遞增參數 B 來計算下一條曲線的點數，直到找出質數階的橢圓曲線參數。

圖 2 為實現 SEA 演算法的 Windows 視窗化

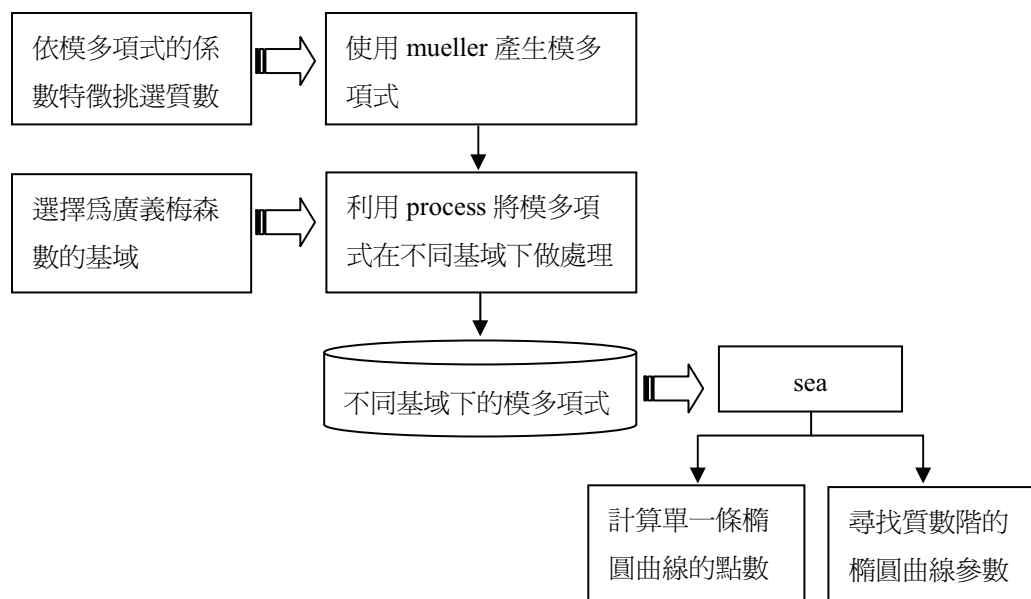


圖 1 系統實作流程圖

表 7 實作的基域總表[1]

編號	基域
p160-1	$2^{160} - 47$
p160-2	$2^{160} + 7$
p192-1	$2^{192} - 237$
p192-2	$2^{192} + 133$
p192-3	$2^{192} - 2^{16} - 1$
p192-4	$2^{192} - 2^{64} - 1$
p192-5	$2^{192} + 2^{120} - 1$
p192-6	$2^{192} + 2^{184} - 1$
p224-1	$2^{224} - 2^{96} + 1$
p256-1	$2^{256} - 189$
p256-2	$2^{256} + 297$
p256-3	$2^{256} + 2^{96} - 1$
p256-4	$2^{256} - 2^{168} + 1$
p256-5	$2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$
p384-1	$2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$
p521-1	$2^{521} - 1$

介面，此系統有二項主要功能，一為就單一組參數「計算橢圓曲線的點數」(圖 3 為執行畫面)，二為使用提早終止策略「遞增 B 來尋找安全的橢圓曲線」(圖 4 為執行畫面)，此為本系統最主要的功能。在計算開始前，使用者需依自己的需求選擇所需的基域，而表 7 裡的 16 個基域所需的模多項式皆已完成前置處理，可以供使用者直接使用，若使用者沒有做選擇，則系統會預設編號為 p192-4 的基域 $p = 2^{192} - 2^{64} - 1$ 。

3.3 實作數據

圖 2 中系統執行一次僅能尋找到一組質數階的橢圓曲線參數，為了能大量記錄系統執行的數據，將圖 2 的介面修改為圖 5，讓使用者可以自行設定所欲尋找的質數階橢圓曲線數量，並在尋找到質數階的曲線參數後自動將每筆運算結果儲存下來，之後供使用者使用。

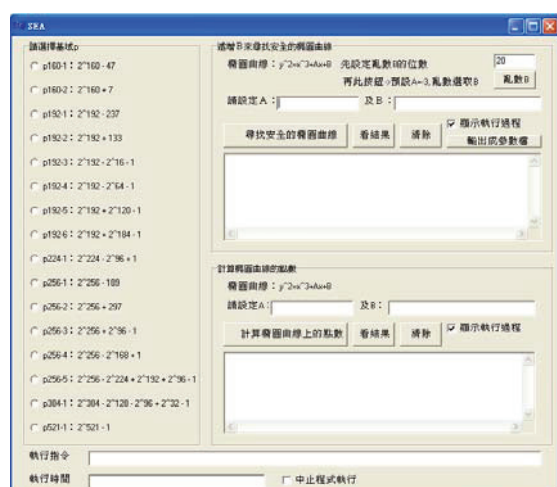


圖 2 SEA 演算法的 Windows 視窗化介面

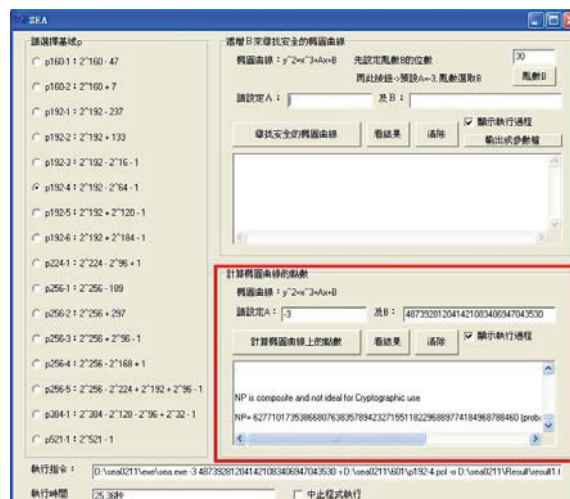


圖 3 「計算單一條橢圓曲線的點數」畫面

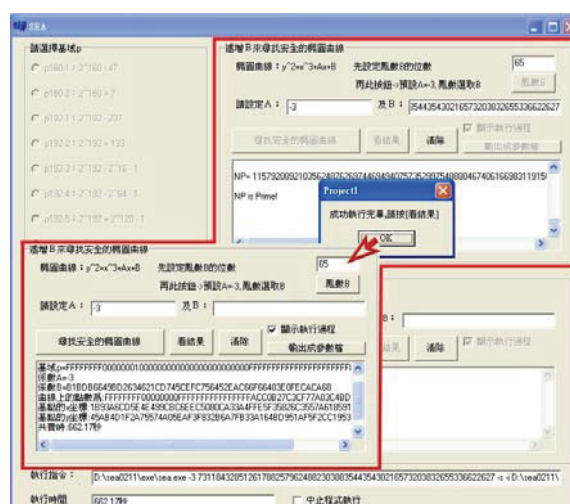


圖 4 「遞增 B 來尋找安全的橢圓曲線」畫面



圖 5 測試系統效能的執行畫面

實驗的軟體設備為 Pentium 1.83G 的處理器搭配 1G 記憶體和 Windows XP 作業系統，從實作的基域(表 7) 中挑選 11 個基域來測量系統執行的效能，表 9 是個別基域實測數據資料整理，再依位元整理實測數據資料如表 8。

符號說明：

N_s ：找到安全橢圓曲線的數量

N_A ：全部搜尋過的橢圓曲線數量

N_T ：實際計算出橢圓曲線點數的數量
 $= N_A - \text{提早終止的曲線數量}$

N_1 ：找到一條質數階橢圓曲線的平均搜尋量
 $= N_A / N_s$

P ：實際計算出橢圓曲線點數的比例
 $= (N_T / N_A) \times 100\%$

T_A ：總執行時間

T_s ：找到一條質數階橢圓曲線的平均時間
 $= T_A / N_s$

T_1 ：計算一條橢圓曲線點數的平均時間
 $= T_A / N_T$

T_p ：計算一條質數階橢圓曲線點數的平均時間

因為採取提早終止策略，所以並不會實際計算出所有曲線的階數，故總執行時間 T_A 中包含提早終止的曲線計算所花費的時間，也就是 T_1 和實際計算一條橢圓曲線點數的時間會有些

微差距，為求精確再針對每一條尋找到的質數階橢圓曲線測量其計算時間，求得平均數 T_p ，並繪出 T_1 和 T_p 的折線圖(如圖 6)，並在圖 7 中加入 T_s 的資訊，最後圖 8 是平均搜尋量 N_1 的折線圖。

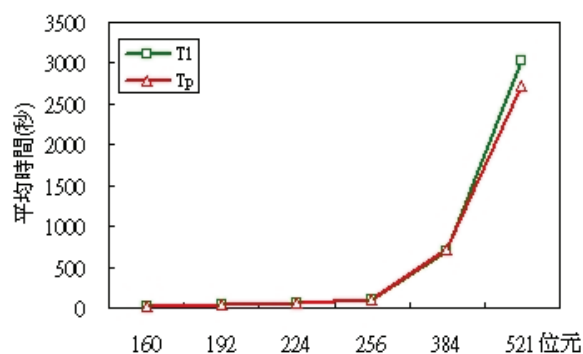


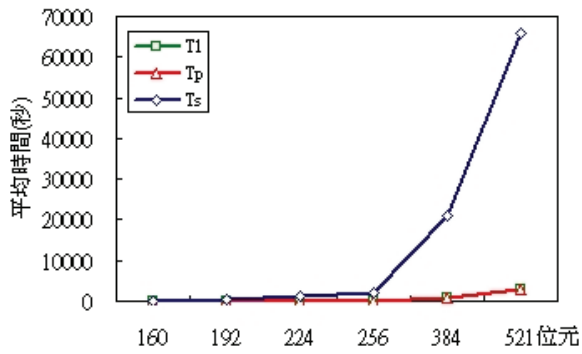
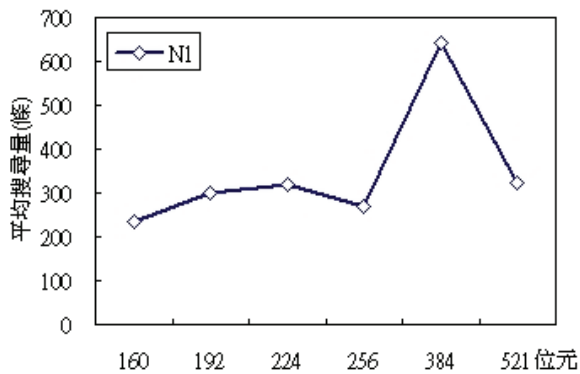
圖 6 T_1 和 T_p 的平均時間折線圖

表 8 個別位元實測數據資料

	位元	160	192	224	256	384	521
N_s	找到安全橢圓曲線的數量	200	300	80	90	22	11
N_A	全部搜尋的橢圓曲線總數量	47066	89472	25677	24238	14164	3560
N_T	實際計算出點數的橢圓曲線數量	2756	5322	1719	1577	653	239
N_1	找到一條安全橢圓曲線的平均搜尋量	235	298	321	269	644	324
P	實際計算橢圓曲線點數的比例(%)	5.86	5.95	6.69	6.51	4.61	6.71
T_A	總執行時間(小時)	11.3	48.4	30.2	47.1	127.8	201.2
T_s	找到一條安全橢圓曲線的時間(秒)	203	580	1360	1884	20907	65862
T_1	計算一條橢圓曲線點數的時間(秒)	14.7	32.7	63.3	107.5	704.4	3031.3
T_p	計算一條質數階橢圓曲線點數的時間(秒)	15.7	31.6	56.7	99.4	722.3	2726.0

表 9 個別基域實測數據資料

編號	基域	N_s	N_A	N_T	N_1	P	T_A	T_s	T_1	T_p
160-1	$2^{160} - 47$	100	21607	1281	216	5.9	5.4	196	15.3	16.1
160-2	$2^{160} + 7$	100	25459	1475	255	5.8	5.8	209	14.2	15.3
192-1	$2^{192} - 237$	100	32195	1849	322	5.7	15.6	563	30.4	29.6
192-4	$2^{192} - 2^{64} - 1$	100	32962	1624	330	4.9	16.0	576	35.5	33.1
192-5	$2^{192} + 2^{120} - 1$	100	24315	1849	243	7.6	16.7	602	32.6	32.1
224-1	$2^{224} - 2^{96} + 1$	80	25677	1719	321	6.7	30.2	1360	63.3	56.7
256-2	$2^{256} + 297$	30	8364	514	279	6.2	14.9	1793	104.6	98.7
256-4	$2^{256} - 2^{168} + 1$	30	7887	522	263	6.6	15.8	1899	109.1	100.9
256-5	$2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$	30	7987	541	266	6.8	16.3	1960	108.7	98.7
384-1	$2^{384} - 2^{128} - 2^{96} + 2^{32} - 1$	22	14164	653	644	4.6	127.8	20907	704.4	722.3
521-1	$2^{521} - 1$	11	3560	239	324	6.7	201.2	65862	3031.3	2726.0

圖 7 T_1 、 T_p 和 T_s 的平均時間折線圖圖 8 平均搜尋量 N_1 折線圖

3.4 橢圓曲線參數

最後提供二組由 SEA 演算法實作系統所產生的橢圓曲線參數：

```

bit = 256
FFFFFFFF000000010000000000000000
p = 00000000FFFFFFFFFFFFFFFFFFFFFFF
FFF
a = -3
b = 313968CBB59979895C5FF778388649
A56E0D354B0EC10927FC964F
FFFFFFFF000000010000000000000000
order = 0397625507521E6BFF994CA2F6E4D5
B0B
7B96E1ABEB3CE68752EB96F896A6
x = F36065C8A17755BB2DFA7F61E3397
F0F4350
91009FB89F90C63D2C7EB8D914EA
y = FE0864715EDF5CF199E35F03F88452
131AB2

bit = 521
1FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
p = FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFF
a = -3

```

```

49A0D9103E590616E8BA10EAE095B
b = FD995EC608B41D7A1272643F660EC
5D999ECA D66D52E6F9CE54607
1FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFFFFFFFFFFFFFFFFFFFFFFFFFFF
FFFFFFF
order = F00B1860FB16FCB5BCF4EA20AD69
A41BCB45E7A0C424AE9E039E5D61
81DBF8D843
F2A157F0D0B5FC2165F7D13920EC
A3006B6FA69B36EB1B6723E7E7DA
x = B446ACF673E919F5EFD945E5B192C
2F7E14A4A95E55C53EEA8838E5668
0FD1CE935BB15FD1
31B45055B9DE0928E03533E6FED15
D587934D06FD29CEE699535C9A96
y = A5DAC1BC9BCFA5FEF4659DCB27F
6F001220D883DC02D6BD2F82E0D2
CF30501FA97F420588

```

4. 結論

橢圓曲線密碼系統的發展已相當成熟，雖然美國國家標準技術局(NIST)已在 FIPS 186-2 標準文件裡推薦了五組質數體的橢圓曲線參數，不過若能讓使用者自行隨機產生所需的橢圓曲線參數，則能達到更佳的安全性，因此實現 SEA 演算法來尋找質數階的橢圓曲線參數對於建立安全的橢圓曲線密碼系統是非常重要的。

在提升實作的速率上可以將仿射座標轉換到投射座標或 Jacobian 座標上，以及令參數 $a = -3$ ，都可以減少點加或倍點的運算量，另外使用提早終止策略，在明確知道曲線的點數不是質數時，直接略過而不計算出其點數，最後可以使用為廣義梅森數的基域，來提升運算的效率。

本研究設計一個實現 SEA 演算法的 Windows 視窗介面，在 Pentium 1.83G 的處理器上搭配 1G 記憶體尋找可供橢圓曲線密碼系統使用的質數階橢圓曲線參數，基域為 192、224、256、384、521 位元平均分別需要 580、1360、1884、20907、65862 秒可搜尋到一條質數階的橢圓曲線參數，且計算一條橢圓曲線的階數平均分別需要 31.6、56.7、99.4、722.3、2726.0 秒。

參考文獻

- [1] 王學理、斐定一，橢圓與超橢圓曲線公鑰密碼的理論與實現，北京：科學出版社，2006 年。
- [2] 任禮祥，橢圓曲線密碼系統之研究與實作，國立中山大學資訊工程學系碩士論文，2005 年。
- [3] 祝躍飛、張亞娟，橢圓曲線公鑰密碼導引，

- 北京：科學出版社，2006年。
- [4] 祝躍飛、顧純祥、裴定一，SEA 算法的有效實現，軟件學報，第 13 卷，第 6 期，2002 年。
 - [5] 楊中皇，網路安全理論與實務，台北市：金禾資訊，2006 年。
 - [6] 董軍武、胡磊、裴定一，Schoof-Elkies-Atkin 演算法的有效實現，Communications of the CCISA，第 12 卷，第 2 期，2006 年 4 月。
 - [7] 董軍武、鄒候文、裴定一，橢圓曲線軟件及密碼卡的設計與實現，計算機應用，第 25 卷，第 11 期，2005 年 11 月。
 - [8] I. Blake, G. Seroussi and N. Smart, *Elliptic Curves in Cryptography*, London Mathematical Society Lecture Note Series 265, Cambridge University Press, 1999.
 - [9] IEEE Standard specifications for public-key cryptography, Jan. 2000, <http://grouper.ieee.org/groups/1363/>.
 - [10] N. Koblitz, "Elliptic Curve Cryptosystems," *Mathematics of Computation*, Vol. 48, No. 177, 1987, pp. 203-209.
 - [11] V. Miller, "Use of Elliptic Curves in Cryptography," volume 218 of *Lecture Notes in Computer Science*, Berlin:Springer-Verlag, 1986, pp. 417-426.
 - [12] MIRACL(Multiprecision Integer and Rational Arithmetic C/C++ Library), Jan. 2008, <http://www.shamus.ie/>.
 - [13] R. Schoof, "Counting points on Elliptic Curves over Finite Fields," *Journal de Theorie des Nombres de Bordeaux* 7, 1995, pp 219-254.
 - [14] J. Solinas, "Generalized Mersenne Numbers," *CACR Technical Report CORR* 99-39, 1999.